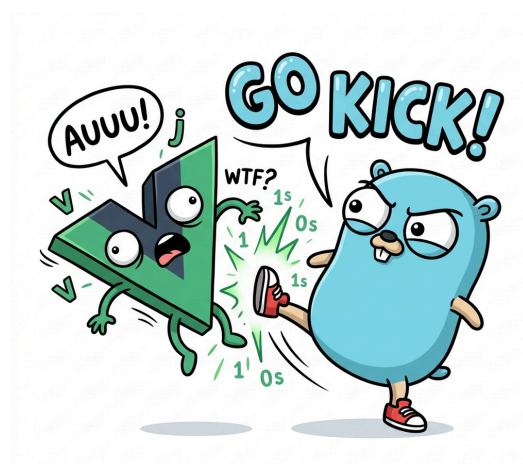


Jak dobrý, výkonný a bezpečný je gokick?

Nezávislý audit reálného kódu (nikoli dokumentace) napříč bezpečností, architekturou, výkonem, dokumentací i frontendem — se srovnáním vůči špičkovým frameworkům.



Souhrnný verdikt. Stack výrazně přesahuje úroveň běžného boilerplate — po architektonické i bezpečnostní stránce jde o nadprůměrné dílo, které v řadě disciplín převyšuje výchozí nastavení mainstreamových frameworků (Rails, Laravel, NestJS). Jediným zásadním stropem je vědomě zvolená **single-node SQLite** — optimální pro latenci a provozní jednoduchost, bez horizontálního škálování.

8,5 / 10 celkové vážené skóre

PŘEDMĚT

Go DDD/CQRS + Vue 3 SPA

ROZSAH KÓDU

20 476 řádků (302 souborů)

METODA

3× hloubkový audit reálného kódu

DATUM

17. 6. 2026

01 Shrnutí & skóre

Hodnocení vzniklo ze tří nezávislých auditů, které procházely **skutečný kód** – bezpečnostní vrstvu, architekturu a CQRS sběrnici, výkonové stropy SQLite, job frontu, tooling i frontend. Skóre a váhy po dimenzích níže.

Jak se počítá celkové skóre

Vážený průměr = $\Sigma(\text{skóre} \times \text{váha}) / \Sigma(\text{vah}) = 64,0 / 7,5 = 8,5$. Všechny dimenze mají váhu $\times 1,0$; jediná výjimka je **škálovatelnost** $\times 0,5$ – vědomě úzký strop, který by s plnou vahou zkresloval celkový obraz. Prostý nevážený průměr by byl 8,25.

Bezpečnost auth, token, logout, hlavičky, audit		9,0/10	$\times 1,0$
Architektura & čistota kódu DDD, CQRS, fail-closed permissions		9,0/10	$\times 1,0$
Tooling / DX / statická vynutitelnost lint, arch-lint, compile-time DI		9,5/10	$\times 1,0$
Výkon – latency, single-node Go runtime + in-proc SQLite		8,5/10	$\times 1,0$
Frontend Vue 3, striktní TS, bezpečný auth model		8,5/10	$\times 1,0$
Dokumentace & AI skills 28 gk-* skillů + štíhlé Documan docs		10/10	$\times 1,0$
Testovací pokrytí 2:1 LOC, integračně těžké, chybí E2E/load		7,5/10	$\times 1,0$
Škálovatelnost – horizontální vědomý strop, ne pro multi-node		4,0/10	$\times 0,5$

Největší síla

Bezpečnostní vrstva na úrovni dedikovaného security týmu: rotace refresh tokenů s **detekcí krádeže**, atomický account lockout, timing-safe login, kompletní CSP/HSTS, audit mimo transakci. Přísnější než výchozí nastavení velkých frameworků.

Největší limit

Single-node SQLite (single-writer, ~stovky–nizké tisíce zápisů/s) a scheduler **bez leader election** (na více instancích běží cron joby opakovaně). Nejde o vadu, nýbrž o záměrné rozhodnutí – jediná binárka, žádná externí infrastruktura.

Jak číst tento dokument

Kapitoly 2–9 rozebírají jednotlivé dimenze s konkrétními důkazy z kódu. Kapitola 10 zasazuje gokick do kontextu Rails/Laravel/Spring/NestJS, kapitola 11 vymezuje **konkrétní cestu k 10/10** v jednotlivých disciplínách, kapitola 12 přináší kvantitativní data o rozsahu kódu.

02 Bezpečnost

9,0 / 10 – to nejsilnější

Bezpečnostní vrstva převyšuje výchozí nastavení Laravelu, Railsu i Expressu. Bez kritických zranitelností; patrné promyšlené threat modelování.

Hesla & login

- bcrypt cost 12 + **SHA-256 prehash** kvůli 72bytovému limitu bcryptu
- Proti unknown useru se počítá **dummy hash** → žádná timing user-enumeration
- Lock check až **po** Verify → časování nic neprozradí

Refresh tokeny

- crypto/rand, v DB jen **SHA-256 hash** (ne plaintext)
- **Rotace + detekce krádeže**: reuse i concurrent-race → odhlášení všech zařízení
- HS256 access token (15 min), refresh 7 dní

Account logout

- 5 pokusů / 10 min → 15 min lock
- Celé **atomicky v SQL** (`UPDATE ... RETURNING`), bez race
- Raw pool → přežije rollback business transakce

HTTP hardening

- CSRF přes **stdlib Go 1.25** (`Sec-Fetch-Site`)
- Kompletní hlavičky: **CSP, HSTS, COOP/CORP**, Permissions-Policy
- Per-IP rate limit (token bucket), maskování tajemství v logu

Nadstandardní opatření

Detekce **reuse rotovaného tokenu** i **concurrent-rotation race** spustí `theft_detected` a tvrdé odhlášení ze všech zařízení. 100 % parametrizované dotazy, JWT secret ≥ 32 znaků vynuceno, `alg`-confusion ošetřen. Audit log běží **mimo transakci**, takže bezpečnostní eventy (login_failed, account_locked) přežijí rollback.

Díličí poznámky (nikoli zranitelnosti): refresh token má ~119 bitů entropie (dostačující); HS256 je symetrický – vyhovující pro jednu službu, pro multi-service nasazení by byl vhodnější asymetrický RS256.

03 Architektura & kvalita kódu

9,0 / 10 – senior-level

DDD + CQRS jsou zde implementované **reálně, nikoli jako deklarace na papíře**. Vzory jsou funkční, pokryté testy a vynucené na více úrovních.

4 vrstvy vynucené `go-arch-lint`

Bounded kontexty (`user` / `token` / `job`) jsou izolované tak, že cross-context import **neprojde lintem**. Doménová vrstva importuje jen stdlib + uuid. Pravidlo závislosti `presentation → application → domain ← infrastructure` je vynucené staticky, ne jen domluvené.

CQRS sběrnice

- Generická, type-safe (`bus.Exec[R]`), bez reflexe
- Audit **mimo** tx, DispatchEvents **obaluje** tx → eventy až po commitu (outbox-like)
- Korektnost **dokázaná** integračními testy

Fail-closed permissions

- Command bez deklarace `Permissioned` / `SkipPermission` **spadne na sběrnici**
- Bezpečnější než spoléhání na nezapomenutý middleware
- Jeden zdroj pravdy odvozený z kódu (BE i FE)

Statically vynucené logování – jedna cesta

depguard allow-list + forbidigo + sloglint (no-raw-keys) znamená, že fyzicky **nelze** propašovat `fmt.Println`, druhý logger ani raw-string klíč. Korelace přes `trace_id` / `user_id`. Takovou disciplínu nenajdeme téměř nikde – ani ve velkých enterprise codebase.

Vyvážená výhrada

Vůči reálnému rozsahu funkcí (v zásadě autentizace + správa uživatelů, ~4,9 k řádků produkčního Go) jde o **značnou míru ceremonie** – vysoký poměr abstrakce k funkcionalitě. To je legitimní za předpokladu, že cílem je pevný základ pro růst. Registry event/job handlerů jsou zatím prázdné – infrastruktura bez odběratelů.

04 Výkon

8,5 / 10 – výborný na jednom uzlu

Na jednom uzlu je výkon výborný – Go runtime bez overheadu spravovaných běhových prostředí a SQLite přímo v procesu, bez síťového hopu.

Latence / single-node

- Go runtime: žádný JVM/PHP/Node overhead
- SQLite **in-process, bez síťového hopu**
- WAL + `_txlock=immediate` (ošetřený `BUSY_SNAPSHOT`)
- Partial indexy přesně na claim-query joby

Job fronta – nadprůměrně robustní

- Atomický `claim` přes `UPDATE...RETURNING`, exponenciální backoff (5 s → cap 1 h)
- At-least-once: job v tx, panika → rollback → znovu claimovatelné, panika do Sentry
- Bez externího brokeru; korektnost na úrovni River/Asynq (jen ~1 s polling)

Jediné výkonové omezení na jednom uzlu

Chybí stránkování – `FindAll` načítá celou tabulku. To je v pořádku do nižších desítek tisíc řádků, dál vyžaduje keyset stránkování. Jinak žádné N+1 ani horké dotazy bez indexu. Horizontální strop (více uzlů, vyšší zápisový throughput) řeší samostatně následující kapitola.

05 Škálovatelnost

4,0 / 10 – vědomý strop

Horizontální škálování je jediný zásadní strop celého stacku – a je vědomě zvolený. Detailní cesta k jeho odstranění je v kapitole 11.

Kde je strop

- SQLite single-writer (~stovky–nízké tisíce zápisů/s)
- Scheduler **bez leader election** → na více instancích běží cron joby opakovaně
- Job fronta = polling po 1 s (ne LISTEN/NOTIFY)
- Rate-limit stav je in-proc → instance nejsou plně stateless

Záměrné rozhodnutí – řeší se výměnou adapteru, ne přepisem

Jediná binárka + volume, žádný Redis/Postgres/broker – ideální pro Dokploy a malá až střední nasazení. Protože perzistence sedí za doménovými `Repository` interface, není strop přepis aplikace, ale **výměna adapteru**.

Preferovaná cesta: adaptér na Postgres (+ River pro joby, leader election pro scheduler). **Alternativa se SQLite (HA + read-scale):** Turso/libSQL (nově i concurrent writes), rqlite, dqlite. Konkrétní kroky v kapitole 11.

06 Tooling / DX

9,5 / 10 — špička

Statická disciplína

- golangci-lint: depguard **allow-list** (každý import schválen), forbidigo, sloglint
- go-arch-lint: vrstvy i bounded kontexty
- ESLint strictTypeChecked + vue-tsc, golines

Build & DI

- Wire = **compile-time DI**, žádná runtime magie
- Goose migrace embedované, auto-aplikace při startu
- Jediný `make` target provede celý build

07 Frontend

8,5 / 10

Bezpečný auth model

- Access token **jen v paměti** (nelze odcizit přes XSS)
- Refresh v HttpOnly cookie
- **Single-flight refresh** + self-healing session

Čistota & typy

- Vue 3 `script setup`, maximálně striktní TS
- Router guardy **zrcadlí backend permissions**
- Žádná FE validace — backend je autoritativní

Stav je řešen **záměrně lehce** — manuální reactive refs plně pokrývají současný rozsah aplikace. Plnohodnotný state management (Pinia) je volbou pro budoucí růst, nikoli chybějícím prvkem.

08 Dokumentace & AI skills

10 / 10 – referenční

Dokumentace je vědomě dvouvrstvá: **štíhlé lidské docs** (Documan) + **28 gk-* skillů** jako strojově i lidsky čitelný how-to. Jde o AI-native přístup, který je v současné praxi vzácný – proto byl vyčleněn jako samostatná hodnocená dimenze.

28 gk-* skillů

- Jeden skill = jeden koncept (auth, repos, jobs, permissions, observability, deploy...)
- Každý: **proč/kdy** · **vysvětlení** · **jak funguje** · **recept** · **časté chyby**
- Psané pro Claude Code i juniora / non-tech čtenáře

Jednotný vstup `/gk`

- Rozcestník `/gk` nebo přímo `/gk-<téma>`
- AI si skill vybere sama podle `description`
- Šablona `_TEMPLATE.md` vynucuje strukturu (6 sekcí, CZ tělo / EN nadpisy)

Štíhlé lidské docs (Documan)

- architecture · installation · configuration · flow stránky · roadmap
- Verbose per-layer docs **vědomě odstraněny** ve prospěch skillů
- Méně driftu, jeden zdroj pravdy pro „jak se co dělá“

`CLAUDE.md` jako kontrakt

- Invarianty, checklist featur, pravidla vrstev na jednom místě
- Drží AI agenta i člověka ve stejných mantinelech
- Mapováno 1:1 na lint/arch pravidla – dokumentace = vynucení

Hodnocení dimenze

Dokumentace primárně určená pro **AI agenta**, již zároveň bez problémů přečte junior, je v současné praxi vzácná – většina projektů spoléhá na zastaralou wiki, nebo nemá nic. Zde je navíc provázaná s kódem přes lint a arch pravidla. ADRs jsou součástí šablony, nikoli tohoto projektu, a proto se do hodnocení nepromítají – dimenze je naplněna v plném rozsahu.

09 Testovací pokrytí

7,5 / 10 – silná šířka, mezery v E2E

Pokrytí je svým rozsahem nadprůměrné a integračně těžké; nejvíc chybí browserové E2E a zátěžové testy.

Co je solidní

- Poměr **testy:produkce u Go** $\approx 2,15:1$ (10 559 vs 4 906 řádků)
- **testfx harness** nad reálnou SQLite – ne mocky
- Arch-konformní testy (`zz_audit` / `zz_gap`) + běh s `-race`
- Regresní test ke každé fázi (event isolation, tx-rollback, claim race...)

Co chybí do 10

- **E2E** (Playwright) – žádné browserové flow testy
- **Zátěžové / výkonové** testy (k6, vegeta)
- Frontend testy (vitest) jsou mělké – mount checks, ne chování
- Bez **coverage gate** v CI a bez mutation/contract testů

Hodnocení dimenze

CI (`validate.yml`) na každý PR spouští lint + test + build, takže regrese se chytají brzy. Slabší skóre nedělá kvalita existujících testů, ale **chybějící vrstvy** – browserová a zátěžová – které u boilerplate teprve dávají jistotu při růstu. Doplnění je přímočaré (kapitola 11).

10 Srovnání s TOP řešeními

ŘEŠENÍ	GOKICK VYHRÁVÁ NA	GOKICK PROHRÁVÁ NA
Rails / Laravel / Django	type-safety, startup, paměť, single-binary deploy, přísnější bezpečnostní defaulty	ekosystém, šíře hotových funkcí, scale-out příběh, komunita
Spring Boot / NestJS	žádný runtime overhead JVM/Node, compile-time DI bez magie za běhu	obří ekosystém integrací, hotové enterprise moduly
Typický Go web projekt	v horních jednotkách procent – disciplína, jakou má málokdo	– (medián je plochý <code>handler + sqlc</code>)

Celkové zarámování

Velké frameworky stojí na principu „baterie + magie + komunita“; gokick na principu „nula magie, plná kontrola, ručně udržované“. Go runtime přináší výbornou latenci a nízkou paměťovou stopu oproti JVM/PHP/Python/Node; pro read-heavy nebo středně zápisové workloady na jednom uzlu je rychlejší a úspornější než kterýkoli z velkých frameworků.

Jako foundation/boilerplate jde prakticky o best-in-class řešení pro malá až střední nasazení s důrazem na bezpečnost, čistotu a jednoduchý provoz. Pro multi-node nebo vysoký zápisový throughput určen není – a ani si to neklade za cíl.

Silné stránky

Bezpečnostní vrstva nad rámec defaultů frameworků.

Reálné, testované DDD/CQRS s lint-enforcementem.

Poměr testy:produkce $\approx 2,15:1$.

Single-binary deploy, embedované migrace i SPA.

11 Cesta k 10/10

Co konkrétně chybí do plného skóre v jednotlivých disciplínách. **Jedna disciplína nese ~90 % práce** – škálovatelnost; zbytek představují cílené dílčí úpravy.

Škálovatelnost 4 → 10 – sem patří většina práce

A) Zůstat u SQLite – HA + read-scale

- **Turso / libSQL** – embedded replicas, nově i concurrent writes (MVCC)
- **rqlite** – Raft, nejzralejší clustering; **dqlite** (Canonical)
- LiteFS je funkční, ale pre-1.0 a deprioritizovaný → pro nové projekty vhodnější Turso

B) Skutečný write-scale – Postgres

- **Výměna adapteru, ne rewrite** – `infrastructure/postgres/*` + `wire.Bind`
- Job frontu → **River** (Postgres-native, battle-tested)
- Scheduler → **leader election** (advisory locks); rate-limit stav → Redis

Bezpečnost 9 → 10

- 2FA (TOTP) + WebAuthn
- refresh 256-bit + EdDSA JWT pro multi-service
- CSP bez `unsafe-inline` (nonce); gosec/CodeQL/govulncheck v CI

Architektura 9 → 10

- zapojit prázdné event/job registry reálnými handlers
- druhý bounded context jako referenční vzor

Tooling / DX 9,5 → 10

- coverage gate + mutation testing v CI
- Dependabot/Renovate + govulncheck
- SBOM + podepsané release (cosign)

Výkon 8,5 → 10

- keyset stránkování list dotazů
- cache vrstva pro hot reads
- benchmarky + pprof profily

Frontend 8,5 → 10

- E2E (Playwright)
- a11y audit (axe)
- i18n

Testovací pokrytí 7,5 → 10

- E2E + load (k6/vegeta)
- coverage gate v CI
- contract + mutation testy

Výsledek

Mimo škálovatelnost jsou všechny dimenze otázkou **týdnů cílené práce**, nikoli přestavby. Architektura tento růst již předpokládá.

12 Statistiky kódu

Měřeno nástrojem `cloc`. Sloupec „kód“ je čistý kód bez komentářů i prázdných řádků.

8 240 řádků

PRODUKČNÍ KÓD

Go 4 906 · frontend 3 255 · SQL 66 · generovaný DI 216

12 030 řádků

TESTOVACÍ KÓD

Go 10 559 · TypeScript 1 471 – poměr ≈ 2,15:1

SLOŽENÍ PRODUKČNÍHO KÓDU



Go (backend) Vue komponenty TypeScript / SQL / generovaný DI

KATEGORIE	SOUBORY	KÓD	KOMENTÁŘE	PRÁZDNÉ
Go – produkce (bez testů a wire_gen)	97	4 906	1 046	899
Go – testy (*_test.go)	86	10 559	1 849	1 699
Go – generovaný (wire_gen.go)	1	216	38	25
Vue (komponenty + views)	47	2 379	14	271
TypeScript – produkce	47	875	208	158
TypeScript – testy	18	1 471	238	464
SQL (migrace)	4	66	18	9
CSS	1	1	0	0
Celkem	302	20 476	3 412	3 527

Co z čísel plyne

Poměr **testy:produkce u Go** ≈ 2,15:1 je silný signál kvality. Hustota komentářů v produkčním Go ≈ 18 % – komentáře jsou „load-bearing“ (vysvětlují proč), ne výplň. Reálná byznys logika je malá (~4,9 k Go) vs. velký aparát kolem – potvrzuje to charakter **foundation/boilerplate postaveného na růst**.

13 Závěr

Jako **foundation/boilerplate je gokick výjimečný** — po bezpečnostní i architektonické stránce nad úrovní výchozího nastavení velkých frameworků a výrazně nad mediánem Go projektů. Strop je jediný a vědomý: single-node SQLite, bez horizontálního škálování.

Pro malá až střední nasazení s důrazem na **bezpečnost, čistotu a jednoduchý provoz** jde prakticky o best-in-class řešení. Pro multi-node nebo vysoký zápisový throughput stavěný není — a ani si to neklade za cíl.

K tomu **AI-native dokumentace** (28 gk-* skillů) posouvá laťku tam, kde ji většina projektů nemá vůbec. Mimo škálovatelnost je **cesta k 10/10 otázkou týdnů cílené práce** (kapitola 11), nikoli přestavby — architektura tento růst již předpokládá.

8,5 / 10

CELKOVÉ VÁŽENÉ SKÓRE

0

KRITICKÝCH ZRANITELNOSTÍ